

DSA BASICS

TOP-PERFORMER HANDWRITTEN ENGINEERING NOTES

17 core pointers • formulas • code patterns • diagrams • revision sheets

Blue-ink notes • red headings • yellow highlights • notebook ruling

Goal: recognize the pattern → choose the data structure → solve → state Time + Space.

CORE MEMORY HOOKS

- ARRAY → index / position
- HASHMAP → key → value
- HASH SET → seen / unique
- TWO POINTERS → left + right
- FREQUENCY → value → count
- BIG O → growth

DSA BASICS

A Kid-Friendly, Interview-Ready Foundation

17 foundational pointers

Arrays • Hashmaps • Hash Sets • Time Complexity • Space Complexity

Two Pointers • Frequency Counting • Big O and more

Goal: understand the idea first, then recognize the pattern in code.

Complete content • Textbook-style formatting • Unicode-safe diagrams and notation

Contents

1. ARRAYS

2. HASHMAPS

3. HASH SETS

4. TIME COMPLEXITY

5. SPACE COMPLEXITY

6. TWO-POINTER TECHNIQUE

7. FREQUENCY COUNTING

8. BIG O — ALL IMPORTANT TYPES

9. Big O Growth Comparison

10. How to Identify Big O From Code

11. Nested vs Sequential Loops

12. Hashmap + Frequency Counting

13. The Most Important DSA Tradeoff

14. How These Concepts Connect

15. Your DSA Mental Checklist

16. The 7 Patterns You Should Memorize First

17. One Final Example Combining Everything

The complete instructional text follows.

Absolutely. Since this is for your DSA foundation, I'll teach it from zero—as if you've never studied DSA

before—but

I'll still take it to the level you need for coding interviews.

DSA Basics — From Zero

Think of DSA = Data Structures + Algorithms.

Imagine you have a huge toy room.

- Data Structure = how you arrange/store the toys.

- Algorithm = the steps you follow to find, arrange, or manipulate the toys.

For example:

"Find my red car among 1,000 toys."

You could:

1. Look at every toy one by one.
2. Keep toys organized by color.
3. Keep a special list of where red cars are.

Same problem, different strategies → different algorithms and data structures.

1. ARRAYS

1.1 What is an array?

Imagine a row of lockers:

10	20	30	40	50		
----	----	----	----	----	--	--

01234

Each locker has:

- a value
- a position

The position is called an index.

In most programming languages, indexing starts at 0.

So:

```
array = [10, 20, 30, 40, 50]
```

index 0 → 10

index 1 → 20

index 2 → 30

index 3 → 40

index 4 → 50

Python:

```
numbers = [10, 20, 30, 40, 50]
```

```
print(numbers[0])
```

Output:

10

1.2 Why are arrays useful?

Because if you know the position, you can immediately access the item.

```
numbers[3]
```

Python doesn't need to start from 10 and walk through 20, 30, 40.

It can directly access the item at index 3.

That's why:

Array access by index = $O(1)$

We'll understand $O(1)$ properly later.

1.3 Common array operations

Suppose:

```
numbers = [10, 20, 30, 40, 50]
```

Access

```
numbers[2]
```

Result:

30

Complexity:

$O(1)$

Change

```
numbers[2] = 99
```

Now:

[10, 20, 99, 40, 50]

Complexity:

$O(1)$

Search

Suppose you want to find 40.

You might do:

```
for x in numbers:
```

```
    if x == 40:
```

```
        print("Found")
```

Worst case:

10 → no

20 → no

30 → no

40 → YES

If 40 were at the end, you'd inspect everything.

Complexity:

$O(n)$

1.4 Inserting into an array

Suppose:

[10, 20, 30, 40]

You want to insert 25 at position 2.

You need:

[10, 20, 25, 30, 40]

But 30 and 40 need to move.

Before:

10 20 30 40

↓ ↓

After:

10 20 25 30 40

If there are 1 million elements, potentially many elements must move.

Therefore:

Insert in middle → $O(n)$

1.5 Deleting from an array

Same idea.

[10, 20, 30, 40, 50]

delete 30

[10, 20, 40, 50]

40 and 50 may need to shift.

Therefore:

Delete from middle $\rightarrow O(n)$

1.6 Array cheat sheet

Operation Complexity

Access by index $O(1)$

Update by index $O(1)$

Search $O(n)$

Insert at end* $O(1)$ amortized

Delete at end* $O(1)$

Insert beginning $O(n)$

Delete beginning $O(n)$

Insert middle $O(n)$

Delete middle $O(n)$

Access by index | $O(1)$

Update by index | $O(1)$

Search | $O(n)$

Insert at end* | $O(1)$ amortized

Delete at end* | $O(1)$

Insert beginning | $O(n)$

Delete beginning | $O(n)$

Insert middle | $O(n)$

Delete middle | $O(n)$

* For a dynamic array such as Python's list, appending is $O(1)$ amortized.

2. HASHMAPS

This is one of the most important DSA concepts for interviews.

Imagine you have a school.

There are 1,000 students.

You want to know:

"What is Rahul's phone number?"

One approach:

Student 1

Student 2

Student 3

...

Student 1000

Search one by one.

That's slow.

Instead, create a dictionary:

Name → Phone

Rahul → 9876

Anu → 8765

John → 7654

This is essentially a hashmap.

In Python:

```
phone_book = {
```

```
"Rahul": "9876",
```

```
"Anu": "8765",
```

```
"John": "7654"
```

```
}
```

Now:

```
phone_book["Rahul"]
```

returns:

9876

Typically:

Hashmap lookup → $O(1)$ average

2.1 Why is it called a "hash" map?

Imagine:

"Rahul"

goes through a special mathematical machine called a hash function.

Conceptually:

"Rahul"

↓

hash function

↓

some number

↓

bucket/location

So the computer can quickly figure out where the information belongs.

You don't need to understand the implementation yet.

For now remember:

```
Hashmap = key → value
```

Examples:

name → age

word → count

student ID → student

country → capital

product ID → price

2.2 Hashmap example

```
ages = {
```

```
"Alice": 20,
```

```
"Bob": 25,
```

```
"Charlie": 30
```

```
}
```

Get Bob's age:

```
ages["Bob"]
```

Result:

25

Add:

```
ages["David"] = 35
```

Delete:

```
del ages["Bob"]
```

Check:

```
if "Alice" in ages:
```

```
    print("Found")
```

2.3 The most important interview pattern

Suppose:

```
numbers = [5, 8, 2, 9, 3]
```

Question:

Does the array contain 9?

Without hashmap:

```
for x in numbers:
```

```
    if x == 9:
```

```
        return True
```

$O(n)$.

With a hashmap/set, you can often get average $O(1)$ membership checking.

This leads to one of the most important DSA ideas:

Use extra memory to make your algorithm faster.

3. HASH SETS

A hashmap stores:

KEY → VALUE

A set stores only unique values.

Think of a box:

{Apple, Banana, Orange}

You don't care about extra information.

You only care:

"Is Apple present?"

Python:

```
fruits = {"apple", "banana", "orange"}
```

Check:

"apple" in fruits

Result:

True

3.1 Sets don't allow duplicates

```
numbers = {1, 2, 3, 3, 3}
```

Becomes:

{1, 2, 3}

3.2 Why are sets useful?

Example

[1, 2, 3, 2, 5, 1]

Question:

Does this array contain duplicates?

We can use a set.

```
seen = set()
```

```
for number in numbers:
```

```
    if number in seen:
```

```
        print("Duplicate!")
```

```
    break
```

```
    seen.add(number)
```

Conceptually:

1 → not seen → add

2 → not seen → add

3 → not seen → add

2 → already seen → DUPLICATE

Complexity:

Time → $O(n)$

Space → $O(n)$

3.3 Hashmap vs Hashset

Remember this:

HASHMAP

key → value

"apple" → 5

"banana" → 3

versus:

HASHSET

value

"apple"

"banana"

"orange"

Use hashmap when:

You need information associated with something.

word → frequency

student → marks

ID → person

Use set when:

You only care whether something exists.

Have I seen this?

Is this unique?

Does this value exist?

4. TIME COMPLEXITY

Now we reach one of the most important DSA concepts.

Imagine two algorithms.

Algorithm A takes:

1 second

Algorithm B takes:

100 seconds

With 10 items, perhaps both are fine.

But what happens with:

1,000,000 items?

This is why we study time complexity.

4.1 What does $O(n)$ mean?

Suppose:

```
for number in numbers:
```

```
    print(number)
```

If:

```
n=5
```

you perform approximately:

5 operations

If:

```
n = 100
```

approximately:

100 operations

If:

```
n = 1,000,000
```

approximately:

1,000,000 operations

So:

$O(n)$

means:

The work grows roughly in proportion to the amount of input.

4.2 Big O does NOT mean exact time

If something is:

$O(n)$

it doesn't mean exactly n seconds or exactly n operations.

Big O describes the growth rate.

For example:

```
for x in numbers:
```

```
print(x)
```

```
for x in numbers:
```

```
print(x)
```

Technically:

$O(2n)$

But we simplify:

$O(n)$

Why?

Because when n becomes enormous, the constant 2 isn't the important part.

5. SPACE COMPLEXITY

Time asks:

How much work?

Space asks:

How much extra memory?

Suppose:

```
def print_numbers(numbers):
```

```
for x in numbers:
```

```
print(x)
```

We aren't creating another array.

So auxiliary space is approximately:

$O(1)$

But:

```
def duplicate(numbers):
```

```
result = []
```

```
for x in numbers:
```

```
result.append(x)
```

```
return result
```

If input has:

10 items → result has 10

1,000 items → result has 1,000

1,000,000 items → result has 1,000,000

Extra memory grows with n .

Therefore:

Space = $O(n)$

5.1 Time vs Space

Think:

TIME

How much work does the computer do?

SPACE

How much extra memory does the computer need?

Example:

```
seen = set()
```

```
for x in numbers:
```

```
    if x in seen:
```

```
        return True
```

```
seen.add(x)
```

Time:

$O(n)$

Space:

$O(n)$

Why?

We may inspect every element.

And we may store every element in seen.

6. TWO-POINTER TECHNIQUE

This is another extremely important interview pattern.

Imagine two people standing at opposite ends of a line.

→ ←

123456789

One person is called:

left

The other:

right

We move them according to the problem.

6.1 Classic example

Array:

[1, 2, 3, 4, 6]

Question:

Find two numbers that add to 7.

Because the array is sorted.

Start:

left = 1

right = 6

Sum:

1+6=7

Done!

Another example:

[1, 2, 3, 4, 8]

Target:

7

Start:

1+8=9

Too large.

Because array is sorted, move right:

1+4=5

Too small.

Move left:

$2+4=6$

Too small.

Move left:

$3+4=7$

Found!

6.2 Code

```
numbers = [1, 2, 3, 4, 8]
```

```
target = 7
```

```
left = 0
```

```
right = len(numbers) - 1
```

```
while left < right:
```

```
    total = numbers[left] + numbers[right]
```

```
    if total == target:
```

```
        print("Found")
```

```
    break
```

```
    elif total < target:
```

```
        left += 1
```

```
    else:
```

```
        right -= 1
```

Complexity:

Time: $O(n)$

Space: $O(1)$

6.3 Why is it $O(n)$?

Because:

```
left
```

only moves forward.

And:

```
right
```

only moves backward.

Neither pointer repeatedly travels across the array.

Together they make at most approximately n movements.

Therefore:

$O(n)$

6.4 When should you think "two pointers"?

Look for problems involving:

- sorted arrays
- pairs
- removing duplicates
- reversing
- palindrome checking

- comparing both ends
- subarrays
- strings

Typical signal:

"Find two elements..."

or

"Compare the beginning and end..."

Think:

TWO POINTERS.

7. FREQUENCY COUNTING

This is one of the easiest and most useful patterns.

Suppose:

apple

How many times does each letter occur?

a → 1

p → 2

l → 1

e → 1

This is frequency counting.

7.1 Using a hashmap

```
text = "apple"
```

```
frequency = {}
```

```
for char in text:
```

```
    if char not in frequency:
```

```
        frequency[char] = 0
```

```
        frequency[char] += 1
```

Result:

```
{
```

```
"a": 1,
```

```
"p": 2,
```

```
"l": 1,
```

```
"e": 1
```

```
}
```

7.2 Better Python version

```
frequency = {}
```

```
for char in text:
```

```
    frequency[char] = frequency.get(char, 0) + 1
```

This pattern is worth memorizing.

7.3 Why is frequency counting powerful?

Consider:

```
[1, 2, 2, 3, 3, 3, 4]
```

Frequency:

```
1 → 1
```

```
2 → 2
```

$3 \rightarrow 3$

$4 \rightarrow 1$

Now you can answer questions such as:

Which number appears most?

Does any number appear twice?

Are two strings anagrams?

How many duplicates exist?

7.4 Anagram example

Are:

"listen"

and:

"silent"

anagrams?

Count letters.

listen:

$l \rightarrow 1$

$i \rightarrow 1$

$s \rightarrow 1$

$t \rightarrow 1$

$e \rightarrow 1$

$n \rightarrow 1$

silent:

$s \rightarrow 1$

$i \rightarrow 1$

$l \rightarrow 1$

$e \rightarrow 1$

$n \rightarrow 1$

$t \rightarrow 1$

Same frequencies.

Therefore:

YES

8. BIG O — ALL IMPORTANT TYPES

Now let's build your Big O ladder.

From generally fastest growth to slowest growth:

$O(1)$

$O(\log n)$

$O(n)$

$O(n \log n)$

$O(n^2)$

$O(n^3)$

$O(2^n)$

$O(n!)$

The further down you go, the more dangerous the algorithm becomes for large inputs.

8.1 $O(1)$ — Constant

This is beautiful.

The input can be:

10

or:

10,000,000

and the operation still takes approximately the same number of steps.

Example:

```
numbers = [10, 20, 30, 40]
```

```
print(numbers[2])
```

You directly access index 2.

$O(1)$

Think:

One door. I know exactly which door to open.

8.2 $O(\log n)$ — Logarithmic

This is extremely efficient.

The classic example is binary search.

Imagine you're guessing a number between 1 and 100.

I say:

Is it 50?

If no:

Is it greater or smaller than 50?

Suppose smaller:

Now:

1–49

Guess:

25

Again, you eliminate approximately half.

Then:

1–24

Then:

1–12

Then:

1–6

Every step cuts the search space approximately in half.

That's:

$O(\log n)$

Binary search example

Sorted:

[10, 20, 30, 40, 50, 60, 70]

Find:

60

Start middle:

40

60 is greater.

Ignore:

10 20 30 40

Now:

50 60 70

Middle:

60

Found.

Only a few steps.

8.3 $O(n)$ — Linear

We already saw this.

```
for x in numbers:
```

```
    print(x)
```

If there are:

100 items $\rightarrow$ ~ 100 iterations

1,000 items $\rightarrow$ $\sim 1,000$

1,000,000 $\rightarrow$ $\sim 1,000,000$

Therefore:

$O(n)$

Think:

Check every child in the classroom.

8.4 $O(n \log n)$

This appears frequently in efficient sorting algorithms.

Examples include:

- Merge sort
- Heap sort
- average/worst-case behavior of some other comparison sorts depending on implementation

Think:

n work

$\times$

$\log n$ levels

For:

```
n = 1,000
```

roughly:

$1,000 \times 10$

operations at the growth-rate level.

So:

$O(n \log n)$

is usually considered very good for sorting large datasets.

8.5 $O(n^2)$ — Quadratic

This usually comes from nested loops.

Example:

```
for i in numbers:
```

```
    for j in numbers:
```

```
        print(i, j)
```

Suppose:

$n=5$

Outer loop:

5 times

Inner loop:

5 times for each outer iteration

Total:

$$5 \times 5 = 25$$

For:

$n = 100$

you get:

$$100 \times 100 = 10,000$$

For:

$n = 1,000,000$

you get roughly:

$$1,000,000,000,000$$

That's enormous.

Therefore:

$O(n^2)$

Think:

Every child must meet every other child.

8.6 $O(n^3)$ – Cubic

Three nested loops:

for i in numbers:

for j in numbers:

for k in numbers:

print(i, j, k)

That's:

$$n \times n \times n$$

Therefore:

$O(n^3)$

For:

$n = 10$

$$10^3 = 1,000$$

For:

$n = 100$

$$100^3 = 1,000,000$$

For:

$n = 1,000$

$$1,000^3 = 1,000,000,000$$

Usually you want to avoid cubic algorithms when n is large.

8.7 $O(2^n)$ — Exponential

Now things get dangerous.

Imagine every person has two choices:

YES

NO

For one person:

2 choices

Two people:

$$2 \times 2 = 4$$

Three:

$$2 \times 2 \times 2 = 8$$

Four:

16

Therefore:

$$2^n$$

This appears in problems involving:

- generating all subsets
- some brute-force recursive algorithms
- certain decision problems

Example:

$$n = 10$$

$$\rightarrow 1,024$$

$$n = 20$$

$$\rightarrow 1,048,576$$

$$n = 30$$

$$\rightarrow 1,073,741,824$$

It grows very quickly.

8.8 $O(n!)$ — Factorial

This is even worse.

Imagine you have:

3 people:

ABC

How many different arrangements?

ABC

ACB

BAC

BCA

CAB

CBA

That's:

$$3! = 6$$

For 4:

$$4! = 24$$

For 5:

$$5! = 120$$

For 10:

$10! = 3,628,800$

For 15:

$15! = 1,307,674,368,000$

This often appears when generating all permutations through brute force.

9. Big O Growth Comparison

This is extremely important to memorize conceptually:

Big O Name Example

$O(1)$ Constant Array index access

$O(\log n)$ Logarithmic Binary search

$O(n)$ Linear One loop

$O(n \log n)$ Linearithmic Merge sort

$O(n^2)$ Quadratic Two nested loops

$O(n^3)$ Cubic Three nested loops

$O(2^n)$ Exponential All subsets

$O(n!)$ Factorial All permutations

$O(1)$ | Constant | Array index access

$O(\log n)$ | Logarithmic | Binary search

$O(n)$ | Linear | One loop

$O(n \log n)$ | Linearithmic | Merge sort

$O(n^2)$ | Quadratic | Two nested loops

$O(n^3)$ | Cubic | Three nested loops

$O(2^n)$ | Exponential | All subsets

$O(n!)$ | Factorial | All permutations

The growth ranking:

SLOWER

↑

$O(n!)$

$O(2^n)$

$O(n^3)$

$O(n^2)$

$O(n \log n)$

$O(n)$

$O(\log n)$

$O(1)$

↓

FASTER

10. How to Identify Big O From Code

This is the skill you need to develop.

Example 1

```
print(numbers[0])
```

Answer:

$O(1)$

Example 2

```
for x in numbers:
```

```
    print(x)
```

Answer:

$O(n)$

One loop.

Example 3

```
for i in numbers:
```

```
    for j in numbers:
```

```
        print(i, j)
```

Answer:

$O(n^2)$

Two nested loops.

Example 4

```
for x in numbers:
```

```
    print(x)
```

```
for y in numbers:
```

```
    print(y)
```

This is:

$O(n) + O(n)$

Therefore:

$O(2n)$

Drop the constant:

$O(n)$

Important: Sequential loops don't automatically become $O(n^2)$.

11. Nested vs Sequential Loops

This is a common beginner mistake.

Nested

```
for i in numbers:
```

```
    for j in numbers:
```

...

$n \times n$

$= O(n^2)$

Sequential

```
for i in numbers:
```

...

```
for j in numbers:
```

...

$n+n$

$= 2n$

$= O(n)$

Remember:

Nested $\rightarrow$ multiply.

Sequential $\rightarrow$ add.

Then simplify.

12. Hashmap + Frequency Counting

Now let's combine concepts.

Question:

Find the first number that appears twice.

Input:

[4, 7, 2, 7, 9]

Use a set:

```
seen = set()
```

```
for number in numbers:
```

```
if number in seen:
```

```
print(number)
```

```
break
```

```
seen.add(number)
```

Walk through it:

4 $\rightarrow$ seen = {4}

7 $\rightarrow$ seen = {4, 7}

2 $\rightarrow$ seen = {4, 7, 2}

7 $\rightarrow$ already seen!

Answer:

7

Complexity:

Time: $O(n)$

Space: $O(n)$

13. The Most Important DSA Tradeoff

Suppose you want to detect duplicates.

Method 1 — Brute force

Compare every number with every other number.

$O(n^2)$

Method 2 — Set

Store previously seen numbers.

$O(n)$

But we use extra memory:

$O(n)$ space

So:

Without extra memory:

$O(n^2)$ time

$O(1)$ space

With a set:

$O(n)$ time

$O(n)$ space

This is called a:

Time-Space Tradeoff

You are essentially saying:

"I'll use more memory so that my program can work faster."

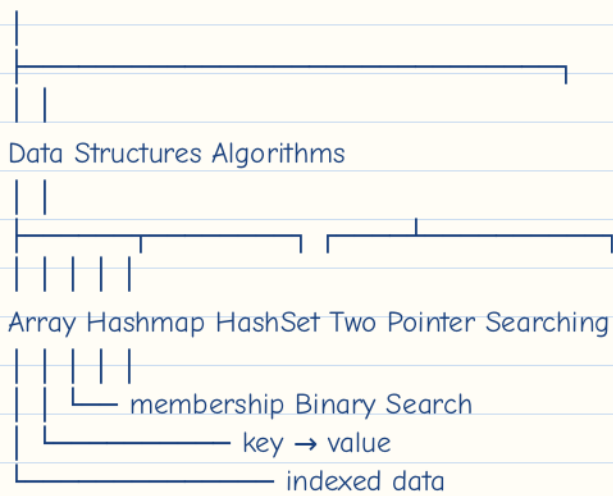
This is one of the fundamental ideas behind DSA.

14. How These Concepts Connect

You should not learn these as eight isolated topics.

They connect like this:

DSA



Frequency Counting

Time Complexity

Space Complexity

15. Your DSA Mental Checklist

When you receive a coding problem, don't immediately start coding.

Ask these questions:

Step 1 — What data do I have?

Array?

String?

Numbers?

Pairs?

Step 2 — Do I need fast lookup?

Think:

HashMap

HashSet

Step 3 — Do I need counts?

Think:

Frequency hashmap

Step 4 — Is the array sorted?

Think:

Binary search

Two pointers

Step 5 — Am I repeatedly searching?

Think:

Can a hashmap/set make this $O(1)$ average?

Step 6 — Am I using nested loops?

Ask:

Can I reduce $O(n^2)$ to $O(n)$?

Step 7 — How much memory am I using?

Calculate:

Space complexity

Step 8 — What's my final complexity?

Always be able to say:

Time: $O(\dots)$

Space: $O(\dots)$

16. The 7 Patterns You Should Memorize

First

For your current DSA preparation, I want these associations to become automatic:

ARRAY

↓

Index / position

$O(1)$ access

HASHMAP

↓

Key → Value

Fast lookup

Frequency counting

HASH SET

↓

Unique values

"Have I seen this?"

Fast membership

TWO POINTERS

↓

left + right

Sorted arrays

Pairs

Palindrome

FREQUENCY COUNTING

↓

value → count

Hashmap

Duplicates

Anagrams

TIME COMPLEXITY

↓

How much work?

SPACE COMPLEXITY

↓

How much extra memory?

And finally:

BIG O

$O(1)$ → excellent

$O(\log n)$ → excellent

$O(n)$ → usually excellent

$O(n \log n)$ → usually good

$O(n^2)$ → potentially expensive

$O(n^3)$ → usually expensive

$O(2^n)$ → dangerous for large inputs

$O(n!)$ → extremely expensive

17. One Final Example Combining Everything

Problem:

Given an array of numbers, determine whether any two numbers add up to a target.

Input:

numbers = [1, 2, 4, 7, 9]

target = 10

Approach 1 — Brute force

Try every pair:

for i in range(len(numbers)):

for j in range(i + 1, len(numbers)):

if numbers[i] + numbers[j] == target:

return True

Two loops:

Time = $O(n^2)$

Space = $O(1)$

Approach 2 — Hash set

```
seen = set()
```

```
for number in numbers:
```

```
    needed = target - number
```

```
    if needed in seen:
```

```
        return True
```

```
    seen.add(number)
```

```
return False
```

Walk through:

```
target = 10
```

```
number = 1
```

```
needed = 9
```

9 not seen

```
seen = {1}
```

```
number = 2
```

```
needed = 8
```

8 not seen

```
seen = {1, 2}
```

```
number = 4
```

```
needed = 6
```

6 not seen

```
seen = {1, 2, 4}
```

```
number = 7
```

```
needed = 3
```

3 not seen

```
seen = {1, 2, 4, 7}
```

```
number = 9
```

```
needed = 1
```

1 IS seen

FOUND!

Complexity:

Time = $O(n)$

Space = $O(n)$

Approach 3 — Two pointers

Because the array is sorted:

12479

↑ ↑

LR

$1 + 9 = 10$

Immediately found.

Complexity:

Time = $O(n)$

Space = $O(1)$

This single problem demonstrates the heart of DSA:

The same problem can have multiple solutions. Your job in an interview is to find an efficient one and explain

why

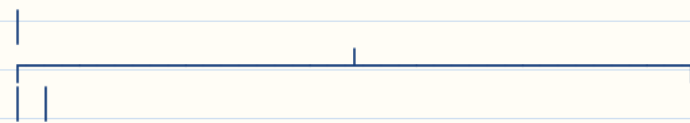
it is efficient.

Your DSA Foundation Map

For your upcoming preparation, I would consider your first DSA foundation layer complete when you can

comfortably explain this without looking at notes:

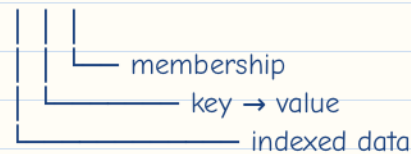
DSA FOUNDATION



DATA STRUCTURES COMPLEXITY



Array HashMap HashSet Time Space



ALGORITHMIC PATTERNS



Two Pointers Frequency Count



left/right value → count

BIG O



$O(1) \rightarrow O(\log n) \rightarrow O(n) \rightarrow O(n \log n)$

$\rightarrow O(n^2)$

$\rightarrow O(n^3)$

$\rightarrow O(2^n)$

$\rightarrow O(n!)$

The key goal is not memorizing definitions. You want to reach the point where you see a problem and your brain

automatically says:

"I need fast lookup → hashmap/set."

"I need counts → frequency map."

"Sorted array + pair → two pointers."

"Halving the search space → binary search → $O(\log n)$."

"Nested loops → probably $O(n^2)$."

"Extra array/set → check space complexity."

That is the foundation we'll build on before moving into harder DSA patterns.

★ FORMULA SHEET

• Array access	$O(1)$
• Array search	$O(n)$
• Middle insert/delete	$O(n)$
• Hashmap lookup	$O(1)$ average
• Hashset membership	$O(1)$ average
• Frequency counting	value $\rightarrow$ count
• Two pointers	Often $O(n)$ time, $O(1)$ extra space
• Binary search	$O(\log n)$
• One loop	$O(n)$
• Two nested loops	$O(n^2)$
• Three nested loops	$O(n^3)$
• Merge sort	$O(n \log n)$
• All subsets	$O(2^n)$
• All permutations	$O(n!)$
• Nested	multiply
• Sequential	add
• Then	simplify constants
• Time	amount of work
• Space	extra memory

★ ONE-PAGE INTERVIEW REVISION

- ARRAY → index, order, contiguous data.
- HASHMAP → key → value, fast lookup, counts.
- HASH SET → unique values, seen-before checks.
- FREQUENCY → value → count.
- TWO POINTERS → sorted array + pair / ends / palindrome.
- BINARY SEARCH → sorted data + eliminate half each step.
- $O(1)$ → constant.
- $O(\log n)$ → halve the search space.
- $O(n)$ → one pass.
- $O(n \log n)$ → efficient sorting / divide-and-conquer growth.
- $O(n^2)$ → two nested loops; look for optimization.
- $O(n^3)$ → three nested loops; usually expensive.
- $O(2^n)$ → all subsets / branching choices.
- $O(n!)$ → all permutations; extremely expensive.

THE INTERVIEW LOOP: Understand problem → identify data → choose pattern → code → test → state
Time + Space → optimize.